

TSFS12 Hand-in 2: Planning for Vehicles with Differential Motion Constraints

MAXENCE MAURY

E-mail: maxma741@student.liu.se

LiU-ID: maxma741

JESSY SANFILIPPO

E-mail: jessa070@student.liu.se

LiU-ID: jessa070

Contents

1	Discussion Topics and Questions	2
1.1	Pure-Pursuit Controller	2
1.2	Linear State Feedback Controller	4
1.3	Non-Linear State Feedback Controller	6

1 Discussion Topics and Questions

1.1 Pure-Pursuit Controller

Exercise 5.1

We need an efficient algorithm to find the pursuit point at distance l along the reference path $p(s)$. A simple pseudo-code algorithm is:

```
function pursuit_point(p_car, path_points, l):
    # p_car: current car position
    # path_points: array of path coordinates
    # l: look-ahead distance

    # Keep track of last closest index for efficiency
    if not initialized:
        idx_start = 0
    else:
        idx_start = last_idx

    for i from idx_start to len(path_points):
        dist = norm(path_points[i] - p_car)
        if dist >= l:
            last_idx = i
            return path_points[i]

    return path_points[-1]    # if no point found, return goal
```

Start from the last closest path point and move forward until a point at distance $\geq l$ from the car is found. Return this point as the pursuit point, or the path's end if none. This avoids costly circle-path intersection checks.

Exercise 5.2

Let pp be the pursuit point in global (world) coordinates, and p_{car} be the car's position in the same frame.

First, we compute the displacement vector from the car to the pursuit point in the global frame:

$$\Delta = pp - p_{\text{car}} = \begin{bmatrix} x_{pp} - x_{\text{car}} \\ y_{pp} - y_{\text{car}} \end{bmatrix}.$$

However, for control purposes, we need the coordinates of the pursuit point in the car's local frame, where the x -axis points forward along the car's heading, and the y -axis points sideways. To convert from world coordinates to the vehicle frame, we rotate Δ by the negative of the car's heading angle θ :

$$\begin{bmatrix} x \\ y \end{bmatrix} = R(-\theta) \Delta, \quad R(-\theta) = \begin{bmatrix} \cos \theta & \sin \theta \\ -\sin \theta & \cos \theta \end{bmatrix}.$$

After this rotation, x represents the forward distance to the pursuit point, and y represents the lateral offset. Notice that $x \neq \Delta_x$ because the car may be oriented at an angle with respect to the global frame.

Given:

```
pp = pursuit point (global coordinates)
pcar = car position (global coordinates)
theta = car heading (global angle)
```

Compute:

```
delta_global = pp - pcar
R = rotation matrix for -theta
delta_local = R @ delta_global
x_local = delta_local[0]
```

Exercise 5.3

We implemented the Pure-Pursuit controller and tested different look-ahead distances l . Results show:

- Small l : very aggressive control, oscillations, risk of instability.
- Large l : smoother control but poor tracking on curves.

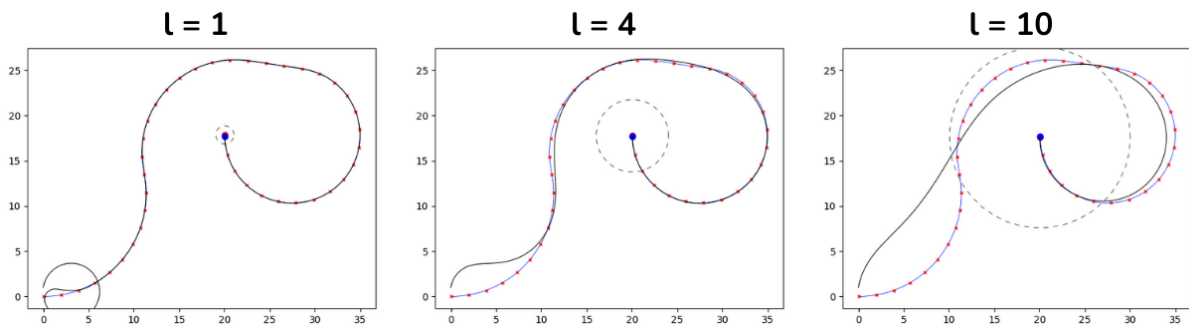


Figure 1: Path tracking with Pure-Pursuit controller for different l .

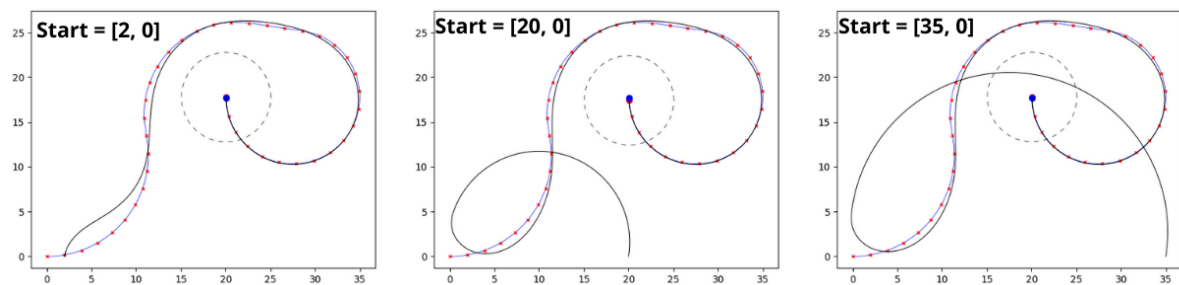


Figure 2: Path tracking with Pure-Pursuit controller for different starting point.

Discussion: Vehicle starts farther from the path than the look-ahead horizon. If the vehicle's initial lateral error is larger than the look-ahead horizon l , the algorithm may fail to find a valid pursuit point along the path within distance l . In this case:

- The controller will typically select the last point on the path or the closest available point, causing the vehicle to steer aggressively towards the path.
- This often leads to overshoot and oscillatory behavior before convergence.
- If no valid point is within l , the vehicle may continue almost straight until the path enters the look-ahead circle, delaying convergence.

A common practical solution is to temporarily increase l when the vehicle is far from the path, or to project the vehicle's position onto the path before applying pure pursuit.

Discussion: Effect of horizon choice. The look-ahead horizon l directly influences the stability-accuracy trade-off:

- Small l : precise path tracking, but sensitive to noise, resulting in oscillations and instability.
- Large l : smoother trajectories and improved stability, but systematic tracking bias in curves (the vehicle "cuts corners").

In practice, an adaptive horizon (smaller l at low speeds and larger l at higher speeds) can balance responsiveness and smoothness.

1.2 Linear State Feedback Controller

Exercise 5.4

The feedback law is

$$u = u_0 - K \begin{bmatrix} d \\ \theta_e \end{bmatrix},$$

where d is the lateral error and θ_e is the heading error. We transform curvature u into steering angle δ by

$$\tan \delta = Lu.$$

The Frenet-frame linearized dynamics around $c(s) = 0$ are:

$$\dot{d} = v\theta_e, \quad \dot{\theta}_e = -u.$$

Using discretization ($T_s = 0.1$ s), the state-space system is

$$x_{k+1} = Ax_k + Bu_k, \quad x = \begin{bmatrix} d \\ \theta_e \end{bmatrix}.$$

Instead of computing the gain K from an LQR design with weighting matrices Q and R , we chose to assign K manually based on empirical tuning. This approach allowed us to directly control the trade-off between lateral error correction and heading stabilization without solving the Riccati equation.

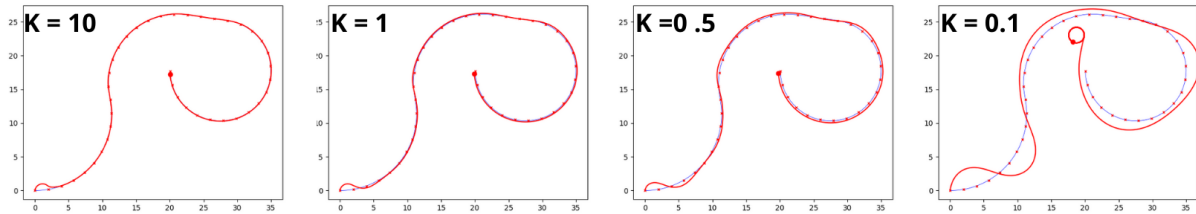


Figure 3: Comparison of Linear Feedback Controller for different values of K .

Exercise 5.5

The projection operation is difficult because it requires solving a nonlinear optimization problem: finding the closest point on a spline path to a given position. In general, there is no closed-form solution, and the projection may not be unique. For example, for a circular path, a point outside the circle has two equally close projections, and in a figure-eight trajectory, a point near the intersection could project onto either loop.

In the *SplinePath* implementation, the projection is approximated numerically. The method defines a function that checks the orthogonality condition between the tangent of the path and the vector from the path to the point. Starting from an initial guess s_0 , it expands a search interval until it finds a sign change, and then uses Brent's root-finding method to solve for the projection parameter s . If no valid projection is found within the limits, it falls back to choosing the closest endpoint.

This approach avoids global optimization, but relies on a good initial guess to converge to the correct solution.

Exercise 5.6

Varying the feedback gains K_1 and K_2 has the following effects:

- **Increasing K_1 (lateral error gain):** The vehicle corrects lateral deviations more aggressively, reducing distance to the path faster. If K_1 is too high, overshoot and oscillations can occur.
- **Increasing K_2 (heading error gain):** The vehicle aligns more quickly with the path direction, which is especially useful when starting far from the path. Excessive K_2 can induce oscillatory heading behavior.
- **Low gains:** With small K_1 and K_2 , convergence to the path is slow, and the vehicle may fail to reach the goal efficiently. If the starting point is far, it can enter a circular motion around the goal.
- **Balance between gains:** A proper ratio ensures stable convergence: typically, a moderate K_1 with a slightly higher K_2 allows heading errors to be corrected before large lateral deviations accumulate.

The stopping distance is controlled by `goal_tol`. If gains are too low or the vehicle starts too far, it may oscillate near the goal before stopping.

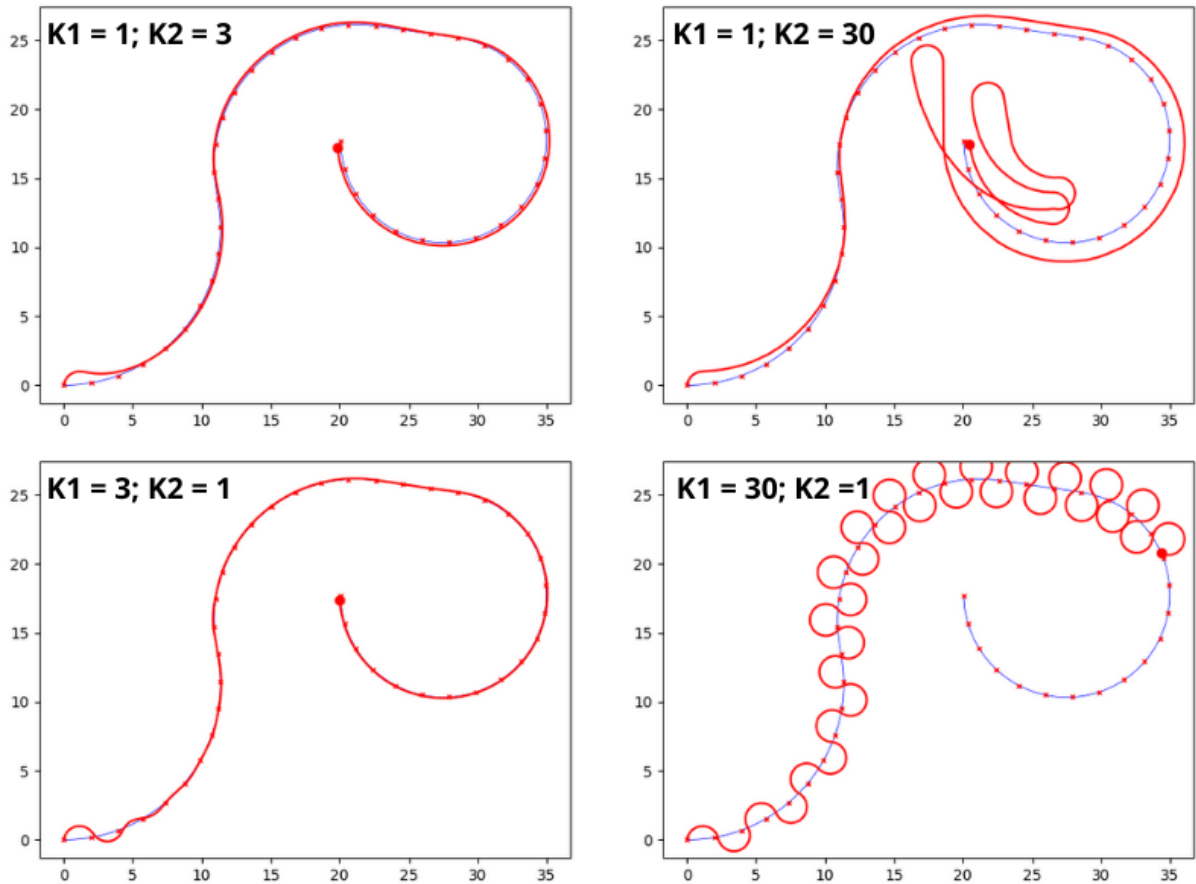


Figure 4: Effect of K_1 and K_2 variations.

1.3 Non-Linear State Feedback Controller

Exercise 5.7

Simulations from $(x, y, \theta, v) = (-10, 10, 0.9\pi/2, 2)$ show that the linear feedback controller struggles when the vehicle starts far from the path. In particular, the vehicle tends to enter a looping or circular motion around the path rather than converging directly. This occurs because the linear controller is designed for small deviations: when the lateral and heading errors are large, the feedback law can produce commands that overcorrect in one direction and then the other, leading to oscillations.

Increasing the heading error gain K_2 relative to the lateral error gain K_1 can mitigate this effect by prioritizing orientation alignment before lateral correction, but the non-linear nature of large errors means the vehicle may still loop if it starts very far from the path. This illustrates the limitation of linear feedback controllers and motivates the use of non-linear feedback laws for large deviations.

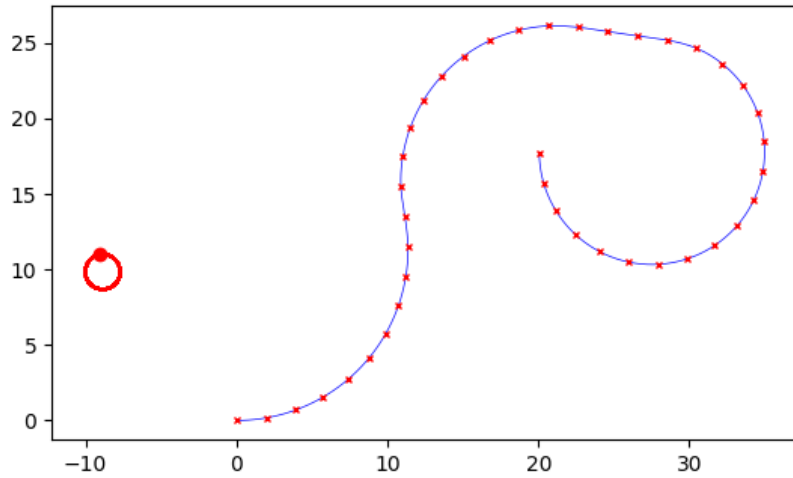


Figure 5: Simulations from $(x, y, \theta, v) = (-10, 10, 0.9\pi/2, 2)$

Exercise 5.8

The nonlinear control law is

$$u = u_0 - K \left[\frac{\sin \theta_e}{\theta_e} d \right].$$

The nonlinear feedback controller modifies the linear law by introducing a scaling factor $\frac{\sin \theta_e}{\theta_e}$, which prevents excessive curvature commands when the heading error θ_e is large. Near $\theta_e = 0$, the series expansion

$$\frac{\sin \theta_e}{\theta_e} = 1 - \frac{\theta_e^2}{6} + \frac{\theta_e^4}{120} - \frac{\theta_e^6}{5040} + O(\theta_e^8)$$

ensures that the control input remains well-defined and smooth.

This non-linear scaling has several effects on vehicle behavior:

- For small heading errors, the controller behaves almost identically to the linear law.
- For large initial errors, the scaling factor reduces the effective lateral gain, preventing the vehicle from overshooting and performing large loops.
- The control signal u is therefore less aggressive during the first transient phase, resulting in smoother convergence to the path.

In practice, this means the vehicle can start farther from the path without exhibiting looping or oscillatory behavior, while maintaining good convergence speed once the vehicle is closer to the path.

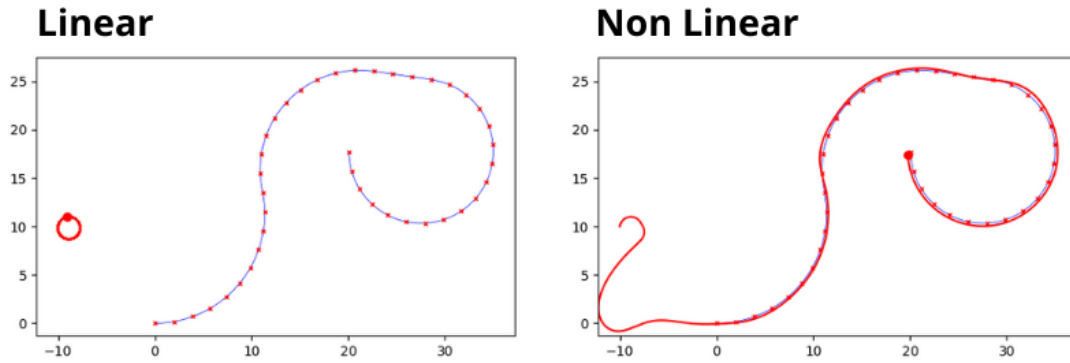


Figure 6: Vehicle trajectory for both the linear and nonlinear controllers.

Exercise 5.9

The comparison between the three controllers highlights their different properties:

- **Pure-pursuit:** This controller is simple and robust to initial errors, but its curvature commands are limited by the geometric look-ahead, leading to slower convergence and less precise path tracking.
- **Linear feedback:** Performs well when the vehicle is close to the path, with precise lateral and heading error reduction. However, when starting far from the path, it can produce oscillations or loops due to aggressive steering commands.
- **Nonlinear feedback:** By scaling the lateral error term with $\frac{\sin \theta_e}{\theta_e}$, this controller avoids excessive curvature commands for large heading errors. It provides smooth convergence even from large initial deviations, while maintaining accurate tracking near the path.

Observations from the plots:

- The trajectory plot shows that the linear controller may overshoot or loop when starting far from the path, whereas the nonlinear controller reaches the path smoothly.
- The control signal plot clearly shows the nonlinear controller limits extreme steering at the beginning of the trajectory, while the linear controller generates higher peaks.
- The path error plot confirms that the nonlinear controller achieves both fast convergence and low steady-state error, combining the advantages of pure-pursuit and linear feedback.

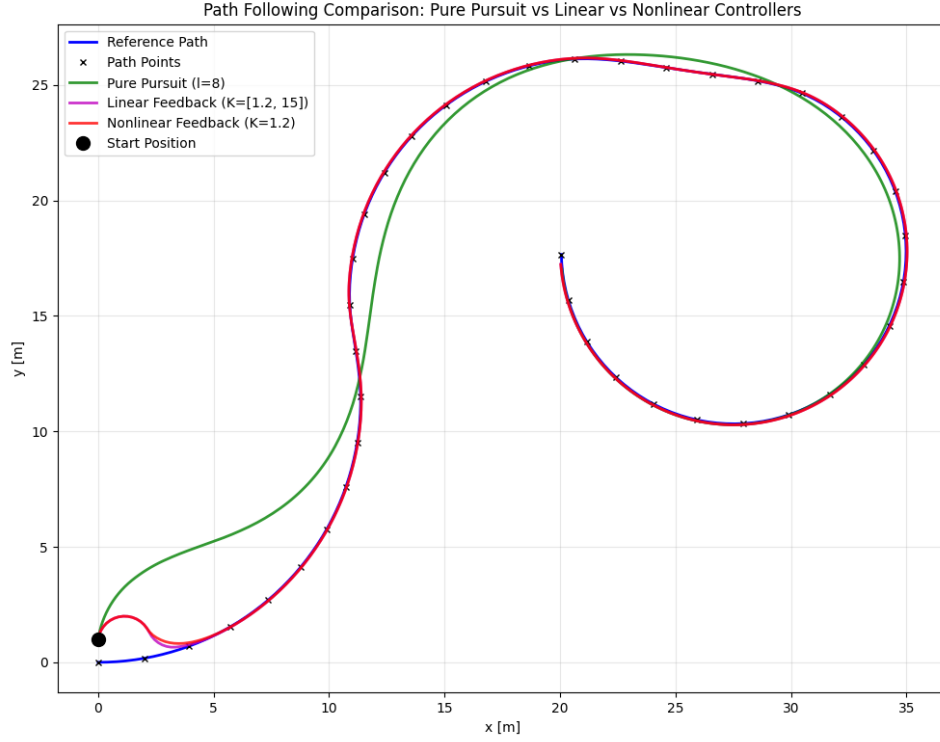


Figure 7: Comparison of path error and control signals for pure-pursuit, linear feedback, and nonlinear feedback controllers starting from $w_0 = (0, 1, 0.9\pi/2, 2)$.

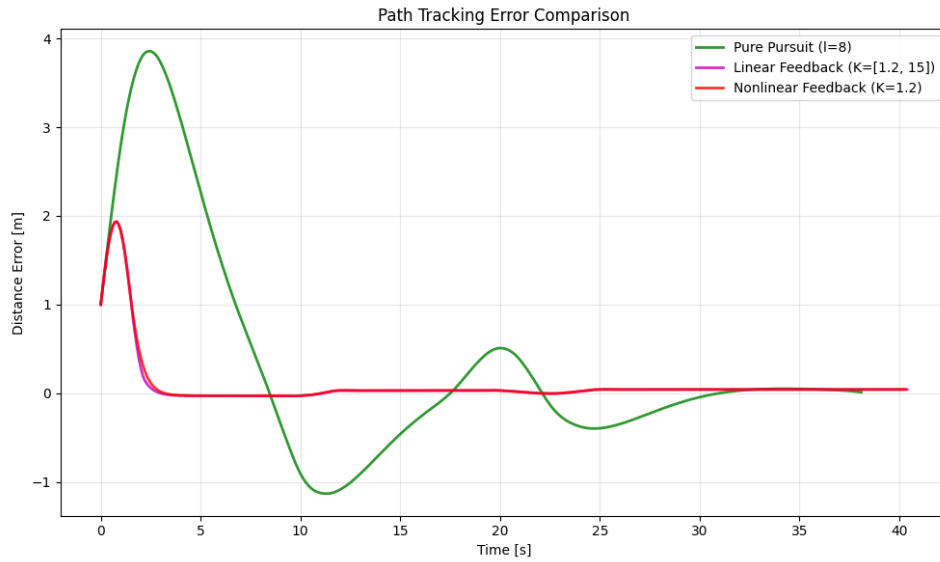


Figure 8: Comparison of distance to path for pure-pursuit, linear feedback, and nonlinear feedback controllers starting from $w_0 = (0, 1, 0.9\pi/2, 2)$.

Conclusion

The study demonstrates that different controllers offer distinct trade-offs between simplicity, precision, and robustness. The pure-pursuit controller is simple and robust to

large initial errors but less precise. Linear feedback provides accurate tracking near the path but can oscillate when starting far away. The nonlinear feedback controller combines the advantages of both, ensuring smooth convergence from large deviations while maintaining precise path tracking.